zeb-index: Benchmarking Autonomous Agents on Production Ready Software Delivery

Framework, Methodology, and a Three-Model Demonstration

Madhavsrikrishna Sriram John Sathya Mohamed Rizwan Sivasaran Chandrasekaran

zeb labs

{madhavsrikrishna.sriram, john.sathya, mohamed.rizwan, sivasaran.sekaran}@zeb.co

Abstract

Choosing a model to build production-ready software needs two answers at once: how accurately it delivers what the business actually needs, and what that accuracy costs. Most coding-agent benchmarks answer neither well, because they score isolated patches against unit tests. *zeb-index* is zeb's benchmark for this decision. It gives a fixed autonomous harness a production-ready project as a sequence of engineering stories that together describe the full business, runs them through a four-phase delivery pipeline – foundation, features, orchestrated integration, and ground-truth judgement – and scores both each story and the integrated result against a reference implementation. Every run is tagged along analytical axes and priced from raw token counts, so accuracy can be read alongside efficiency and cost.

We demonstrate zeb-index on 23 production-ready projects (136 stories) with three frontier models: Claude Sonnet 5, Kimi-K3, and GLM-5.3. The benchmark separates what a single score would hide. Story-level accuracy is nearly tied (0.848–0.870); against the reference implementation every model drops by roughly a quarter of a point (0.592–0.643), and the best model changes with tier and workload. Cost spans a 2.3× range per story, so cost varies far more than accuracy.

published by zeb.co

1 Introduction

Teams adopting autonomous coding agents are not asking which model writes the best isolated function. They are asking which model, inside their harness, can deliver a production-ready system that does what the business needs – and at what cost. Answering that needs a benchmark with three properties:

- **Business-complete work.** The unit of evaluation is a whole project, specified as the engineering stories a team would actually write, so that delivering all of them requires understanding the full business, not one function.
- **Accuracy against a reference, not just against tests.** A delivery is judged on whether it behaves like a reference implementation of the same project, including its security behaviour, not only on whether its own tests pass.
- **Analytics, not a leaderboard.** Results are broken down by project tier, technology stack, and story size, and priced from raw token counts, so accuracy can be weighed against efficiency and cost for the kind of system being built.

zeb-index is built to provide all three. This report describes the framework (§2) and demonstrates it on three frontier models (§3–5).

2 The zeb-index Framework

2.1 Projects and Stories

A zeb-index project is a production-ready software system – a backend service, frontend client, infrastructure estate, ML/AI service, or agentic system – with a reference implementation. Each project is decomposed into engineering stories with business context and acceptance criteria, authored so that, taken together, they describe the whole business the system serves. Every project is tagged with a **delivery-complexity tier** (W1–W4) and a **technology stack**, and every story with an **authored effort** (Fibonacci story points 5, 8, or 13) set before any model runs.

2.2 Harness

Every run drives **Prime Agent**, an RLM (reasoning-language-model) harness that works through a single persistent programming kernel rather than a fixed menu of typed tools. It runs autonomously until it has pushed a branch and opened a pull request, or has hit a hard limit. The harness is held fixed and the model is the swappable component, so any difference in outcome is attributable to the model.

2.3 The Four-Phase Pipeline

Each project is delivered in four phases:

1. **Foundation.** One story that seeds shared scaffolding – configuration, logging, dependency-injection wiring, base error scheme, packaging and health checks – and nothing business-specific. The model's own foundation is merged to `main` and becomes the base for every later story.
2. **Feature.** Every remaining story, each implemented independently and in parallel on its own branch off the model's foundation, without seeing the other features' work. Some features depend on code that a sibling feature delivers; that dependency is not visible while they run.
3. **Orchestrator.** A single integration run per project, driven by the same model. The orchestrator discovers every feature branch, merges them onto `main`, resolves the conflicts between stories that were written without knowledge of each other, and re-checks each story's acceptance criteria after the merge. Its output is one integrated system. Because the features were built in isolation, the orchestrator can miss dependencies between them.
4. **Ground truth.** A judge compares the integrated `main` with the project's reference implementation, end to end.

2.4 Scoring

Both scoring passes use the same agentic judge, Claude Opus 5.5, for every model. The judge reads the committed code and, where possible, builds and runs it.

Story-level judge (Phases 1–2). The judge inspects each story's pull request against its acceptance criteria and combines three rubric dimensions:

$$q(r) \;=\; 0.45\, d_{\text{business}} \;+\; 0.40\, d_{\text{code}} \;+\; 0.15\, d_{\text{tests}}.$$

Each criterion records its evidence and whether it came from execution or reading.

published by zeb.co

Ground-truth judge (Phase 4). The judge derives business scenarios from the reference implementation's behaviour and checks whether the integrated delivery produces the same outcome, however it gets there. It uses the same three dimensions and weights. Structure is deliberately not scored: file layout, endpoint paths, library choice, and module boundaries can all differ from the reference without penalty. Security is a **hard gate**: if the delivery fails to refuse any attack-shaped input that the reference refuses (forged token, cross-account access, injection), the business-logic dimension is capped regardless of everything else, so a security miss cannot be averaged away by strong tests or clean code.

2.5 Analytics Layer

Every run records its turns, tool calls, and full token breakdown (input, output, cache read, cache write). Cost is computed from those raw counts against published per-model rates, never from provider-declared cost fields, so models served on different endpoints are priced on the same basis. Combined with the tier, stack, and effort tags, this gives accuracy, efficiency, and cost side by side for any slice of the benchmark.

3 Demonstration Setup

3.1 Models

Claude Sonnet 5 is served over Amazon Bedrock; Kimi-K3 and GLM-5.3 over Modal's shared inference endpoint (Table 1). All three run at the harness's high reasoning-effort setting under one configuration; only the model varies.

Label	Role	Serving
Claude Sonnet 5	Solver (harness driver)	Amazon Bedrock
Kimi-K3	Solver (harness driver)	Modal shared endpoint
GLM-5.3	Solver (harness driver)	Modal shared endpoint
Claude Opus 5.5	Judge (fixed across all solvers)	Amazon Bedrock

Table 1: Solver models compared, plus the single fixed judge.

3.2 Projects

The demonstration uses 23 production-ready projects spanning Python, Go, TypeScript, JavaScript, C#, and HCL/Terraform, across backend, frontend, infrastructure-as-code, ML/AI, and agentic workloads. Each contributes one foundation story, a variable number of feature stories, one orchestrator run, and one ground-truth comparison. Every model ran the same 23 foundation and 113 feature stories (136 in total) and the same 23 ground-truth comparisons.

3.3 Pricing

Dollar figures are solver cost for the foundation and feature stories, priced with the rates in Table 2. Judge and orchestrator cost are excluded.

published by zeb.co

Model	Input	Output	Cache read	Cache write
Claude Sonnet 5 (solver)	$2.00	$10.00	$0.20	$2.50
Kimi-K3 (solver)	$3.00	$15.00	$0.30	$3.00
GLM-5.3 (solver)	$1.40	$4.40	$0.26	$1.40

Table 2: Published $/million-token rates (Modal shared-endpoint pricing for Kimi-K3 and GLM-5.3; Amazon Bedrock on-demand pricing for Claude Sonnet 5). GLM-5.3's cache write bills at the input rate – moot, since neither Kimi-K3 nor GLM-5.3 ever reports a nonzero cache-write token.

4 Accuracy

4.1 Story-Level Accuracy

At story level the three models are close to a tie, within roughly two hundredths of a point (Table 3). Kimi-K3 has the highest mean score and the fewest turns; GLM-5.3 has the lowest score but also the lowest variance. zeb-index reports every story-level result by tier, stack, and story size as well; in this demonstration no model leads every tier or every stack, and story size predicts cost but not score.

Model	Score	SD	Turns	Cache read	$/story (solver only)
Claude Sonnet 5	0.866	0.082	133	7.75 M	$2.49
Kimi-K3	**0.870**	0.071	**51**	**1.81 M**	$1.58
GLM-5.3	0.848	0.060	61	2.35 M	**$1.08**

Table 3: Story-level results, mean per story (136 stories per model). Bold marks the best value in each column.

4.2 Ground-Truth Accuracy

What this score measures. Feature stories are solved in parallel, each independently of the others, and some depend on code that a sibling feature delivers. The orchestrator merges them afterwards and can miss those cross-feature dependencies, which can pull the ground-truth score below what a fully coordinated build would achieve. The ground-truth score is therefore not an upper bound on what each model could deliver. It measures how well each model's delivery holds up *after orchestration*, with that limitation applied identically to all three models.

Results. Against the reference implementation, every model loses roughly a quarter of a point relative to its story-level score (Table 4, Figure 1): stories that each satisfy their own acceptance criteria do not, once integrated, reproduce the full behaviour of the business. Kimi-K3 keeps the highest mean, and the gap between models widens from 0.022 at story level to 0.051.

Model	Story-level score	Ground-truth score	SD	Drop
Claude Sonnet 5	0.866	0.624	0.170	−0.242
Kimi-K3	**0.870**	**0.643**	0.189	**−0.227**
GLM-5.3	0.848	0.592	0.178	−0.256

Table 4: Ground-truth judge score per model (mean over 23 projects), against the mean story-level score from Table 3. Bold marks the best value in each column.

published by zeb.co

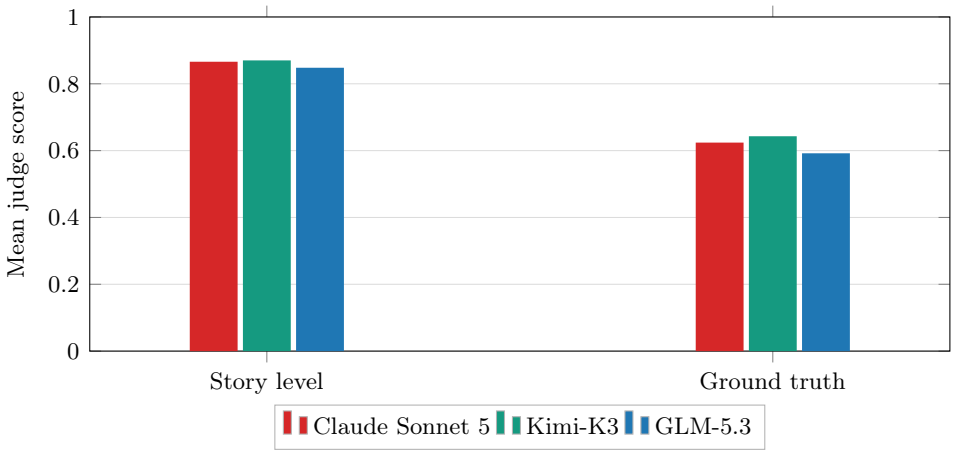


Figure 1: Story-level versus ground-truth score. Every model drops by roughly a quarter of a point when the integrated system is compared with the reference implementation.

A mean hides how differently the models fail (Table 5). Claude Sonnet 5 has a lower mean than Kimi-K3 but posts the best ground-truth score on 12 of the 23 projects, against 7 for Kimi-K3 and 4 for GLM-5.3. Kimi-K3 has the most projects at or above 0.75 and the fewest below 0.5, but also the single lowest score. Under the security hard gate (§2.4), one missed attack case caps business value however complete the rest of the delivery is, which can put a project well below the others.

Model	Median	Min	Max	<0.50	≥0.75	Projects won
Claude Sonnet 5	0.675	0.340	**0.871**	7	8	**12**
Kimi-K3	**0.705**	0.289	0.866	**6**	**9**	7
GLM-5.3	0.602	0.310	**0.871**	8	4	4

Table 5: Distribution of ground-truth scores over the 23 projects. "Projects won" counts the projects where a model has the highest ground-truth score of the three.

By tier and workload the ranking is not stable (Table 6). Claude Sonnet 5 leads at W1, W3, and W4 and on backend, agentic, and full-stack projects; Kimi-K3 leads at W2 and on infrastructure-as-code by a wide margin (0.768 against 0.646 and 0.592); GLM-5.3, the lowest-scoring model overall, leads on frontend and ML/AI projects. Agentic and full-stack projects are the hardest for every model, with no model above 0.53.

published by zeb.co

Axis	Group (n)	Claude Sonnet 5	Kimi-K3	GLM-5.3
Tier	W1 (3)	**0.788**	0.776	0.751
	W2 (6)	0.523	**0.659**	0.559
	W3 (11)	**0.606**	0.579	0.549
	W4 (3)	**0.730**	0.711	0.657
Workload	Agentic (4)	**0.523**	0.517	0.435
	Backend (7)	**0.679**	0.655	0.605
	Frontend (5)	0.626	0.646	**0.685**
	IaC (4)	0.646	**0.768**	0.592
	ML/AI (2)	0.683	0.713	**0.745**
	Full-stack (1)	**0.435**	0.407	0.351

Table 6: Mean ground-truth score by delivery-complexity tier and workload category; n is the number of projects. Bold marks the best value in each row.

5 Efficiency and Cost

Table 7 summarises solver consumption and cost per story. Claude Sonnet 5 processes three to four times as many tokens per story as the other two models, almost all of it cache reads, and costs the most per story. Cost per story spans a 2.3× range and tokens per story a three- to four-fold range, while story-level accuracy differs by about two hundredths of a point and ground-truth accuracy by about five hundredths.

Model	$/story	Tokens/story	Cache read/story
Claude Sonnet 5	$2.49	7.93 M	7.75 M
Kimi-K3	$1.58	**1.92 M**	**1.81 M**
GLM-5.3	**$1.08**	2.49 M	2.35 M

Table 7: Mean solver cost and tokens per story (136 stories per model). Tokens/story is input + output + cache read + cache write. Orchestrator and judge consumption are excluded. Bold marks the lowest value in each column.

6 What zeb-index Shows

The demonstration illustrates what the framework adds over a single-score benchmark.

- **Story-level scores alone overstate readiness.** All three models look strong and nearly tied story by story; only the ground-truth pass shows that the integrated systems fall about a quarter of a point short of the reference, and by different amounts.
- **The right model depends on what is being built.** The model with the best mean is not the one that wins the most projects, and the lowest-mean model leads on frontend and ML/AI work. The per-axis view is what makes that visible.
- **Accuracy and cost have to be read together.** Cost and consumption vary far more than accuracy, so a model choice made on accuracy alone misses most of the difference between the options.

published by zeb.co

7 Scope and Caveats

Story-level results are means over the foundation and feature stories; ground-truth results cover the system produced by the orchestrator, which is not scored or costed separately. Because features run in parallel and the orchestrator can miss cross-feature dependencies, ground-truth scores measure post-orchestration accuracy, not the accuracy of a fully coordinated build. All dollar figures are solver cost only, priced at published rates. The 23 projects vary in scope and are not a balanced grid, and several tier and workload groups contain only one to three projects, so those rows are indicative. All three models ran the identical stories and ground-truth comparisons under the same harness, judge, and controls.

8 Roadmap

This report showcases zeb-index through a focused demonstration with three models – Claude Sonnet 5, Kimi-K3, and GLM-5.3 – on one harness and 23 production-ready projects. Its purpose is to show what the benchmark measures and the analytics it produces, not to rank the full field of models. The next releases of zeb-index will build on it as follows.

- **Frontier-model release.** In an upcoming release, we will publish zeb-index results for all of the latest frontier models on production-ready projects, run under the same pipeline, judge, and pricing method as this report, so that any model can be compared end to end on delivering business-complete, production-ready solutions.
- **Coordinated feature delivery.** Features are currently built in parallel, and the orchestrator can miss dependencies between them (§4.2). We intend to add a dependency-aware variant of the pipeline, so that the gap between post-orchestration and fully coordinated accuracy can be measured directly rather than only noted as a caveat.
- **Broader and more balanced coverage.** Several tier and workload groups in this report contain only one to three projects. Adding projects to those groups will make the per-axis results more robust.

published by zeb.co

9 Conclusions

zeb-index benchmarks autonomous agents on production-ready software delivery: business-complete projects delivered through a four-phase pipeline, scored story by story and against a reference implementation, with accuracy, efficiency, and cost reported by tier, stack, and story size. Applied to Claude Sonnet 5, Kimi-K3, and GLM-5.3, it shows a near-tie at story level, a roughly quarter-point drop against the reference that varies by model and workload, and a much wider spread in cost than in accuracy. Those are the distinctions a team needs when choosing a model to build a production-ready system, and they are the ones a single leaderboard score hides.